FedAdOb: Privacy-Preserving Federated Deep Learning with Adaptive Obfuscation

Hanlin Gu, Jiahuan Luo, Yan Kang, Yuan Yao, *Member, IEEE,* Gongxi Zhu, Bowen Li, Lixin Fan, *Member, IEEE,* Qiang Yang, *Fellow. IEEE*

Abstract—Federated learning (FL) has emerged as a collaborative approach that allows multiple clients to jointly learn a machine learning model without sharing their private data. The concern about privacy leakage, albeit demonstrated under specific conditions [1], has triggered numerous follow-up research in designing powerful attacking methods and effective defending mechanisms aiming to thwart these attacking methods. Nevertheless, privacy-preserving mechanisms employed in these defending methods invariably lead to compromised model performances due to a *fixed* obfuscation applied to private data or gradients. In this article, we, therefore, propose a novel *adaptive* obfuscation mechanism, coined FedAdOb, to protect private data without yielding original model performances. Technically, FedAdOb utilizes passport-based adaptive obfuscation to ensure data privacy in both horizontal and vertical federated learning settings. The privacy-preserving capabilities of FedAdOb, specifically with regard to private features and labels, are theoretically proven through Theorems 1 and 2. Furthermore, extensive experimental evaluations conducted on various datasets and network architectures demonstrate the effectiveness of FedAdOb by manifesting its superior trade-off between privacy preservation and model performance, surpassing existing methods.

Index Terms—Federated learning; privacy-preserving computing; Adaptive Obfuscation; Passport

arXiv:2406.01085v1 [cs.CR] 3 Jun 2024

1 Introduction

Federated Learning (FL) offers a privacy-preserving framework that allows multiple organizations to jointly build global models without disclosing private datasets [2], [3], [4], [5]. Two distinct paradigms have been proposed in the context of FL [5]: Horizontal Federated Learning (HFL) and Vertical Federated Learning (VFL). HFL focuses on scenarios where multiple entities have similar features but different samples. It is suitable for cases where data sources are distributed, such as healthcare institutions contributing patient data for disease prediction. On the other hand, VFL addresses situations where entities hold different attributes or features of the same samples. This approach is useful in scenarios like combining demographic information from banks with call records from telecom companies to predict customer behavior.

Since the introduction of HFL and VFL, studies have highlighted the existence of privacy risks in specific scenarios. For instance, Zhu et al. reported that semi-honest attackers can potentially infer *private features* from the released model and updated gradients [1] in HFL. The leakage risk of *private labels* has also been identified by Jin et al. [6] and Fu et al. [7] in VFL. To mitigate these concerns, a variety of defense methods have been proposed to enhance the privacy-preserving capabilities of Federated Learning. Specifically, for HFL, defense approaches such as differential privacy [8], gradient compression [9], SplitFed [10], and mixup [11], [12] have been put forward. For VFL, defense methods encompass differential privacy techniques like noise addition and random response [7], [13], [14], [15], as well as gradient discretization [16] and gradient sparsification [17]. Nevertheless, our comprehensive analysis and empirical investigation (refer to Sect. 6.2) reveal that all the aforementioned defense mechanisms experience a certain level of performance degradation.

The existing privacy defense methods [7], [18], [16], [17], [9], [11] can be fundamentally understood as obfuscation mechanisms that offer privacy assurances through the utilization of an obfuscation function $g(\cdot)$ applied to private feature (such as transferred model weights in HFL and forward embedding in VFL). Take the HFL as one example (see details for both HFL and VFL in Sect. 4), the process is depicted as follows:

$$x \xrightarrow{g(\cdot)} g(x) \xrightarrow{F_\omega} \ell \longleftarrow y, \tag{1}$$

in which F represents the model parameterized by ω and ℓ is the loss. Noted that in Eq. (1), the level of privacy-preserving capability is determined by the *extent of obfuscation* (the distortion $g()$), which is controlled by a fixed hyperparameter. As examined in previous studies [19], [20], substantial obfuscation inevitably results in the loss of information in $g(x)$ and $g(\omega)$, thereby impacting the model's performance (as observed in the fundamental analysis in Sect. 4.4 and empirical investigation in Sect. 6.2). We contend that a fixed obfuscation strategy fails to consider the dynamic nature of the learning process and serves as the fundamental cause of model performance deterioration.

As a remedy to shortcomings of the fixed obfuscation, we propose to **adapt obfuscation function** g_ω during the learning of

- *Hanlin Gu, Jiahuan Luo, Yan Kang and Lixin Fan are with WeBank AI Lab, WeBank, China. E-mail: {allengu, jihuanluo, yangkang, lixinfan}@webank.com.*
- *Yuan Yao is with the Department of math, , Hong Kong University of Science and Technology, Hong Kong. E-mail: yuany@ust.hk.*
- *Gongxi Zhu is with the School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu, China. E-mail: gx.zhu@foxmail.com.*
- *Bowen Li is with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China. E-mail: libowen@sjtu.edu.cn.*
- *Qiang Yang is with the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, Hong Kong and WeBank AI Lab, WeBank, China. E-mail: qyang@cse.ust.hk.*

Corresponding author: Lixin Fan.

model F_ω, such that the obfuscation itself is also optimized during the learning stage. That is to say, the learning of the obfuscation function also aims to preserve model performance by tweaking model parameters ω:

$$x \xrightarrow{g_\omega(\cdot)} g(x) \xrightarrow{F_\omega} \ell \longleftarrow y, \tag{2}$$

We regard this adaptive obfuscation in both HFL and VFL as the gist of the proposed method, called **FedAdOb**. In this work, we implement the adaptive obfuscation based on the passport technique, which was originally designed for protecting the intellectual property of deep neural networks (DNN) [21], [22]. More specifically, we decompose the entire network into a bottom model, which is responsible to extract features, and a top model which outputs labels. Both the bottom model and top model embed private passports to protect, respectively, privacy of features and labels. The proposed framework FedAdOb has three advantages: i) Private passports embedded in bottom and top layers unknown to adversaries, which prevents attackers from inferring features and labels. It is *exponentially hard* to infer features by launching various attacks, while attackers are defeated by a *non-zero recovery error* when attempting to infer private labels (see Sect. 5). ii) Passport-based obfuscation is learned in tandem with the optimization of model parameters, thereby preserving model performance (see Sect. 4.1 and Sect. 4.4). iii) The learnable obfuscation is efficient, with only minor computational costs incurred since no computationally extensive encryption operations are needed (see Sect. 6.2). Our contributions are summarized as follows:

- We propose a general privacy-preserving FL framework by leveraging the adaptive obfuscation, named FedAdOb, which could simultaneously preserve the model performance, data privacy, and training efficiency. The proposed framework can be applied in the HFL and VFL.
- Theoretical analysis demonstrates that the proposed FedAdOb could protect features and labels against various privacy attacks.
- Extensive empirical evaluations conducted on diverse image and tabular datasets, along with different architecture settings, demonstrate that FedAdOb achieves a trade-off between privacy protection and model performance that is comparable to near-optimal levels, surpassing existing fixed obfuscation defense methods.

2 Related Work

Federated learning has three categories [5] from the perspective of how data is distributed among participating parties:

1) *horizontal federated learning*: datasets share the same feature space but different space in samples;
2) *vertical federated learning*: two datasets share the same sample ID space but differ in feature space;
3) *federated transfer learning*: two datasets differ not only in samples but also in feature space.

We focus on HFL and VFL settings. In the following, we review privacy attacks and defense mechanisms proposed in the literature for HFL and VFL.

2.1 Horizontal Federated Learning

The horizontal federated learning (HFL) was proposed by McMahan et al. [4], aiming to build a machine learning model based on datasets that are distributed across multiple devices [3], [4] without sharing private data with the server and other devices.

Privacy Attacks. There are mainly two types of privacy attacks in HFL: gradient inversion (GI) [1], [23] and model inversion (MI) [24]. GI tries to infer the private data by minimizing the distance between the estimated and observed gradients. MI recovers the private data by comparing the estimated feature output and the observed one.

Defense Mechanisms. Differential privacy (DP) [8], gradient compression (GC) [9], [25], homomorphic encryption (HE) [26], secure multi-party computation (MPC) [27], and mixup [12], [11] are widely used privacy defense mechanisms. The cryptographic techniques HE and MPC can guarantee data privacy but have a high computational and communication expense. DP and GC distort shared model updates to protect privacy, often dramatically worsening model performance. Mixup protects data privacy by mixing-up private images with other images randomly sampled from private and public datasets, which also degrades the model's performance.

2.2 Vertical Federated Learning

Vertical Federated Learning (VFL) is proposed to address enterprises' demands of leveraging features dispersed among multiple parties to achieve better model performance, compared with the model trained by a single party, without jeopardizing data privacy [5]. In VFL, privacy is a paramount concern because participants of VFL typically are companies whose data may contain valuable and sensitive user information. We review privacy attacks and defense mechanisms of VFL as follows.

Privacy Attacks in VFL. Privacy attacks can be categorized into Feature Inference (FI) attacks and Label Inference (LI) attacks in VFL. FI attacks are initiated by the active party and aim to discover the features of the passive party. Model inversion [24] and CAFE [6] are the two most well-known FI attacks in VFL. LI attacks, on the other hand, are initiated by the passive party and aim to infer labels possessed by the active party. LI attacks have two types: gradient-based and model-based. The former [28] calculates the norm or direction of gradients back-propagated to the passive party to determine labels, while the latter [7] first pretrains an attacking model and then leverages this attacking model to infer labels.

Defense Mechanisms. Cryptography-based defense mechanisms such as Homomorphic Encryption (HE) and Multi-Party Computation (MPC) are widely adopted in VFL to protect data privacy for their high capability of preserving privacy. For example, Hardy et al. [26] proposed vertical logistic regression (VLR) using homomorphic encryption (HE) to protect feature privacy, and Zhou et al. [29] proposed the SecureBoost, a VFL version of XGBoost, that leverages HE to protect gradients exchanged among parties. However, Cryptography-based defense mechanisms typically have high computation and communication costs. Thus, they are often applied to shallow models (e.g., LR and decision trees) compared with deep neural networks.

Non-cryptography-based defense mechanisms protect data privacy typically by distorting the model information to be disclosed to adversaries. For example, Differential Privacy [8] adds noise to disclosed model information while Sparsification [7], [9] compresses disclosed one to mitigate privacy leakage. Specialized defense techniques such as MARVELL [28] and Max-Norm [28] are designed to thwart gradient-based label inference attacks by

TABLE 1: Threat model we consider in this work.

Setting	Adversary	Attacking Target	Attacking Method	Adversary's Knowledge
VFL	Semi-honest passive party	*Labels* owned by the active party	PMC [7]	A few labeled samples
			NS [28]	No prior
			DS [28]	No prior
	Semi-honest active party	*Features* owned by a passive party	BMI [24]	Some labeled samples
			WMI [24], [6]	Passive models
HFL	Semi-honest server	*Private data* of clients	WMI [24]	Model parameters and output
			BMI [24]	Model output and a few auxiliary samples
			WGI [1]	Model parameters and gradients
			BGI [1]	Model gradients

applying optimized or heuristic noise to gradients. Additionally, InstaHide [11] and Confusional AutoEncoder [30] are defensive mechanisms that encode private data directly to enhance data privacy.

3 Preliminary

In this section, we introduce the two main federated learning settings: horizontal federated learning and vertical federated learning.

3.1 Horizontal Federated Learning

Consider a Horizontal Federated Learning (HFL) consisting of K clients who collaboratively train a HFL model ω to optimize the following objective:

$$\min_{\omega} \sum_{k=1}^{K} \sum_{i=1}^{n_k} \frac{\ell(F_\omega(x_{k,i}), y_{k,i})}{n_1 + \cdots + n_K}, \tag{3}$$

where ℓ is the loss, e.g., the cross-entropy loss, $\mathcal{D}_k = \{(x_{k,i}, y_{k,i})\}_{i=1}^{n_k}$ is the dataset with size n_k owned by client k.

In each communication round, client k uploads their own model weights ω_k $(k = 1, \cdots, K)$ to the server and then the server aggregates all uploaded model weights through $\omega = \frac{1}{K}\sum_{k=1}^{K} \omega_k$. Next, the server distributes the aggregated weights to all clients (see Fig. 2(a)).

Threat Model: We assume the server might be *semi-honest* adversaries such that they do not submit any malformed messages but may launch *privacy attacks* on exchanged information from other clients to infer clients' private data.

We consider four types of threat models, summarized in Tab. 1: i) White-box Model Inversion (WMI), where the adversary knows the model parameters and feature output to restore private data via model inversion attack [24]; ii) Black-box Model Inversion (BMI), where the adversary only knows feature output and a few auxiliary samples to restore private data via model inversion attack [24]; iii) White-box Gradient Inversion (WGI), where the adversary knows the model parameters and gradients to restore private data via gradient inversion attack [31]; IV) Black-box Gradient Inversion (BGI), where the adversary only knows the model parameters to recover private data via gradient inversion attack [31].

3.2 Vertical Federated Learning

Consider a Vertical Federated Learning (VFL) setting consisting of one active party P_0 and K passive parties $\{P_1, \cdots, P_K\}$ who collaboratively train a VFL model $\Theta = (\theta, \omega)$ to optimize the following objective:

$$\min_{\omega, \theta_1, \cdots, \theta_K} \frac{1}{n} \sum_{i=1}^{n} \ell(F_\omega \circ (G_{\theta_1}(x_{1,i}), G_{\theta_2}(x_{2,i}), \cdots, G_{\theta_K}(x_{K,i})), y_i), \tag{4}$$

where passive party P_k owns features $\mathcal{D}_k = (x_{k,1}, \cdots, x_{k,n}) \in \mathcal{X}_k$ and the passive model G_{θ_k}, the active party owns the labels $y \in \mathcal{Y}$ and active model F_ω, $\mathcal{X}_k$ and $\mathcal{Y}$ are the feature space of party P_k and the label space respectively. Each passive party k transfers its forward embedding H_k to the active party to compute the loss. The active model F_ω and passive models $G_{\theta_k}, k \in \{1, \ldots, K\}$ are trained based on backward gradients (See Fig. 2(b) for illustration). Note that, before training, all parties leverage Private Set Intersection (PSI) protocols to align data records with the same IDs.

Threat Model: We assume all participating parties are *semi-honest* and do not collude with each other. An adversary (i.e., the attacker) $P_k, k = 0, \cdots, K$ faithfully executes the training protocol but may launch privacy attacks to infer the private data (features or labels) for other parties.

We consider two types of threat models summarized in Tab. 1: i) The active party wants to reconstruct the private features of a passive party through the WMI [24] or BMI [6], [24]. ii) A passive party wants to infer the private labels of the active party through the Passive Model Completion (PMC) [7], Norm-based Scoring (NS) function and Direction-based Scoring (DS) function [28].

4 The Proposed Method: FedAdOb

We first introduce the adaptive obfuscation module and then apply this adaptive obfuscation module to the HFL and VFL settings.

4.1 Adaptive Obfuscation Module

This section illustrates two critical steps of the Adaptive Obfuscation Module, i) embedding private passports to adapt obfuscation; ii) generating passports randomly to improve privacy-preserving capability.

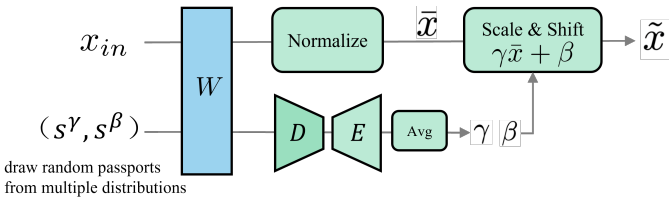


Fig. 1: Adaptive obfuscation ($g_W(\cdot)$). We implement $g_W(\cdot)$ by inserting a passport layer into a normal neural network layer.

4.1.1 Random Passport Generation

How passports are generated is crucial in protecting data privacy. Specifically, when the passports are embedded in a convolution layer or linear layer with c channels[1], for each channel $j \in [c]$, the

1. For the convolution layer, the passport $s \in \mathbb{R}^{c \times h_1 \times h_2}$, where c is channel number, h_1 and h_2 are height and width; for the linear layer, $s \in \mathbb{R}^{c \times h_1}$, where c is channel number, h_1 is height.

passport $s(j)$ (the j_{th} element of vector s) is randomly generated as follows:

$$s(j) \sim \mathcal{N}(\mu_j, \sigma^2), \quad \mu_j \in \mathcal{U}(-N, 0), \tag{5}$$

where all $\mu_j, j = 1, \cdots, c$ are different from each other, $\mathcal{U}$ represents the uniform distribution, σ^2 is the variance of Gaussian distribution and N is the *passport mean range*, which are two crucial parameters of FedAdOb. The strong privacy-preserving capabilities rooted in such a random passport generation strategy are justified by theoretical analysis in Theorems 1 and 2 as well as experiment results in Sect. 6.2.

4.1.2 Embedding Private Passports

In this work, we adopt the DNN passport technique proposed by [32], [21] as an implementation for the adaptive obfuscation framework of FedAdOb. Specifically, the adaptive obfuscation is determined as follows (described in Algo. 1):

$$\begin{aligned} g_W(x_{in}, s) =& \gamma(Wx_{in}) + \beta, \\ \gamma =& \text{Avg}\Big(D\big(E(Ws^\gamma)\big)\Big), \\ \beta =& \text{Avg}\Big(D\big(E(Ws^\beta)\big)\Big), \end{aligned} \tag{6}$$

自适应混淆实现

where W denotes the model parameters of the neural network layer for inserting passports, x_{in} is the input fed to W, γ and β are the scale factor and the bias term. Note that the determination of the crucial parameters γ and β involves the model parameter W with private passports s^γ and s^β, followed by a autoencoder (Encoder E and Decoder D with parameters W') and a average pooling operation Avg$(\cdot)$. Learning adaptive obfuscation formulated in Eq. (6) brings about two desired properties:

- Passport-based parameters γ and β provide strong privacy guarantee (refer to Sect. 5): without knowing passports, it is exponentially hard for the attacker to infer layer input x_{in} from layer output $g_W(x_{in}, s)$, because attacker have no access to γ and β (see Theorem 1).
- Learning adaptive obfuscation formulated in Eq. (6) optimizes the model parameter W through three backpropagation paths via β, γ, W, respectively, which helps preserve model performance. This is essentially equivalent to adapting the obfuscation (parameterized by γ and β) to the model parameter W (more explanations in Sect. 4.4); This adaptive obfuscation scheme offers superior model performance compared to fixed obfuscation schemes (see Sect. 6.2).

Algorithm 1 Adaptive Obfuscation ($g_W(\cdot)$)

Input: Model parameters W of the neural network layer for inserting passports, the input x_{in} to which the adaptive obfuscation applies, passport keys $s = (s^\gamma, s^\beta)$.
Output: The obfuscated version of the input.

Compute $\gamma = \text{Avg}\left(D\big(E(W * s^\gamma)\big)\right)$
Compute $\beta = \text{Avg}\left(D\big(E(W * s^\beta)\big)\right)$
return $\gamma(W * x_{in}) + \beta$

4.2 FedAdOb in HFL

We first partition the neural network model into two components, namely the bottom layers represented by G_θ and the top layers represented by f_ω. In this partitioning, the bottom layers are considered private, while the top layers are shared for aggregation in the federated learning process. Each client participating in the federated learning employs a private adaptive obfuscation technique within the bottom model to protect the client's private feature, as outlined in Eq. (6).

The training procedure of Federated Adaptive Obfuscation (FedAdOb) in the context of horizontal federated learning is depicted in Fig. 2(a). Algo. 2 provides a detailed description of this training procedure.

1) Each client optimizes the adaptive obfuscation g_{θ_k} and federated model f_ω with its private passports s_k and data $\mathcal{D}_k$ according to the cross-entropy loss. Then, all clients upload the federated model ω_k to the server (line 4-8 of Algo. 3);
2) The central server aggregates federated models ω_k to obtain ω [4] and distributes the aggregated model ω to all clients; (line 10-11 of Algo. 3);

The two steps iterate until the performance of the model does not improve.

在HFL中的实现

Algorithm 2 FedAdOb in HFL

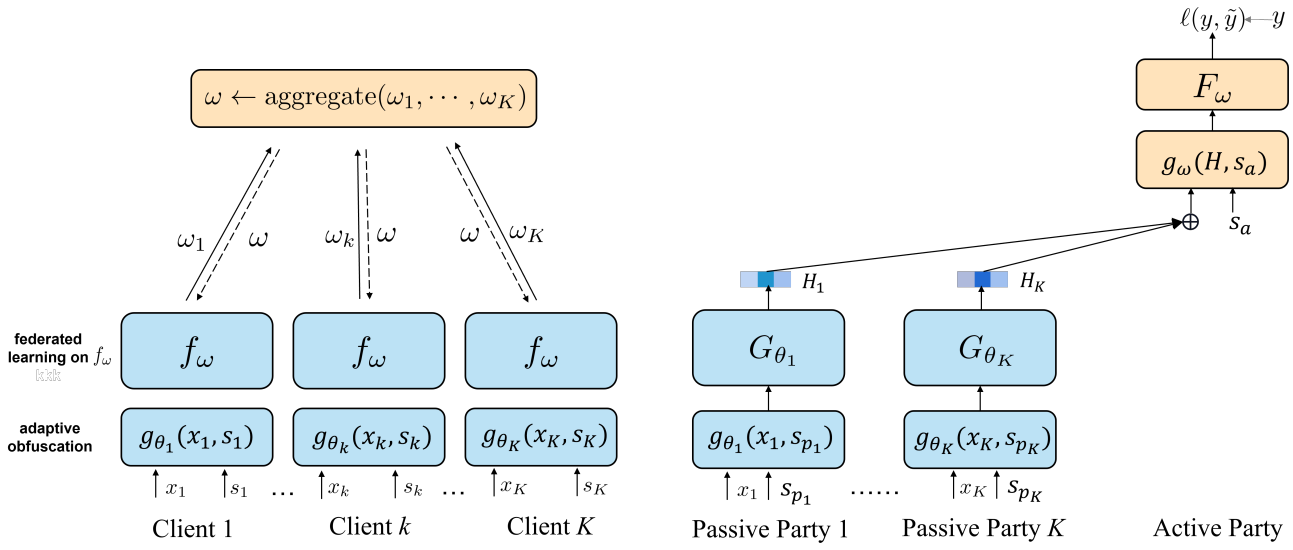
Input: Communication rounds T, # of clients K, learning rate η, the dataset $\mathcal{D}_k = \{x_{k,i}, y_{k,i}\}_{i=1}^{n_k}$ owned by client k and the passport mean range and variance $\{N^k, \sigma^k\}$ for client k.
Output: $\theta_1, \cdots, \theta_K, \omega$

Initialize adaptive obfuscation parameters $\theta_1, \cdots, \theta_K$ and public model parameter ω.
for t in communication round T **do**
  ▷ *Clients perform:*
  for Client k in $\{1, \ldots, K\}$ **do**:
    $\omega_k \longleftarrow \omega$;
    for Batch $(B_x, B_y) \in \mathcal{D}_k$ **do**:
      Sample the passport tuple $s_k = (s_k^\gamma, s_k^\beta)$ via Eq. (5) and N^k, σ^k;
      Compute loss $\tilde{\ell} = \ell(f_{\omega_k} \circ g_{\theta_k}(s_k, B_x), B_y)$;
      $\theta_k \longleftarrow \theta_k - \eta \nabla_{\theta_k} \tilde{\ell}$;
      $\omega_k \longleftarrow \omega_k - \eta \nabla_{\omega_k} \tilde{\ell}$;
    Upload ω_k to the server;
  ▷ *The server performs:*
  The server aggregates: $\omega = \frac{1}{K}(\omega_1 + \cdots + \omega_K)$;
  Distribute ω to all clients;
return $\theta_1, \cdots, \theta_K, \omega$

4.3 FedAdOb in VFL

As discussed in Sect. 3.2, the active parties in Vertical Federated Learning (VFL), who possess the bottom model, are concerned about the potential leakage of labels to passive parties. Conversely, the passive parties, who possess the top model, are keen to safeguard their private features from being reconstructed by the active parties.

To address the simultaneous protection of private features and labels, we introduce the adaptive obfuscation module in VFL, implemented separately for the bottom and top models. This module ensures the simultaneous protection of both labels and features. The framework of Federated Adaptive Obfuscation (FedAdOb) in vertical federated learning is depicted in Fig. 2(b).



(a) HFL (b) VFL

Fig. 2: The left sub-figure illustrates the FedAdOb for HFL, including adaptive obfuscation g_θ and federated model f_ω. The right sub-figure illustrates the FedAdOb for the VFL setting, in which multiple passive parties and one active party collaboratively train a VFL model, where passive parties only have the private features x, whereas the active party has private labels y. Both the active party and the passive party adopt adaptive obfuscation by inserting passports into their models to protect features and labels.

在VFL中的实现

The specific training procedure is illustrated as follows (described in Algo. 3):

1) Each passive party k applies the adaptive obfuscation to its private features with its private passports s_{p_k} and then sends the forward embedding H_k to the active party (line 3-9 of Algo. 3);
2) The active party sums over all $H_k, k \in \{1, \ldots, K\}$ as H, and applies the adaptive obfuscation to H with its private passports s_a, generating $\tilde{H}$. Then, the active party computes the loss $\tilde{\ell}$ and updates its model through back-propagation. Next, the active party computes gradients $\nabla_{H_k}\tilde{\ell}$ for each passive party k and sends $\nabla_{H_k}\tilde{\ell}$ to passive party k (line 10-19 of Algo. 3);
3) Each passive party k updates its model θ_k according to $\nabla_{H_k}\tilde{\ell}$ (line 20-22 of Algo. 3).

The three steps iterate until the performance of the joint model does not improve.

4.4 Update Procedure by Adaptive Obfuscation

Consider a neural network $f_\Theta(x) : \mathcal{X} \to \mathbb{R}$, where $x \in \mathcal{X}$, $\Theta = (\omega, \theta_1, \cdots, \theta_K)$ denotes top model and bottom model parameters. Then we can reformulate the loss of FedAdOb as the following:

Proposition 1. *The loss of the FedAdOb with adaptive obfuscation in the bottom model can be written as:*

- *For VFL:*

$$\min_{\omega,\theta_1,\cdots,\theta_K} \frac{1}{n}\sum_{i=1}^{n} \ell(F_\omega \circ (G'_{\theta_1}(x_{1,i}, s_{p_1})), \cdots, G'_{\theta_K}(x_{K,i}, s_{p_K})), y_i)), \tag{7}$$

- *For HFL:*

$$\min_{\theta_1,\cdots,\theta_K,\omega} \sum_{k=1}^{K}\sum_{i=1}^{n_k} \frac{\ell(F_\omega \circ (G'_{\theta_k}(x_{k,i}, s_{p_k}), y_{k,i})}{n_1 + \cdots + n_K}, \tag{8}$$

Algorithm 3 FedAdOb in VFL

Input: Communication rounds T, passive parties number K, learning rate η, batch size b, the passport mean range and variance $\{N_a, \sigma_a\}$ and $\{N_{p_k}, \sigma_{p_k}\}$ for the active party and passive party k respectively, the feature dataset $\mathcal{X}_k = (x_{k,1}, \cdots, x_{k,n_k})$ owned by passive party k, the aligned label $\mathcal{Y} = (y_1, \cdots, y_{n_0})$ owned by the active party.
Output: Model parameters $\theta_1, \cdots, \theta_K, \omega$

1: Initialize model weights $\theta_1, \cdots, \theta_K, \omega$.
2: **for** t in communication round T **do**
3: ▷ *Passive parties perform:*
4: **for** Passive Party k in $\{1, \ldots, K\}$ **do**:
5: Sample a batch $B_x = (x_{k,1}, \cdots, x_{k,b})$ from the dataset $\mathcal{X}_k$;
6: Sample the passport tuple $s_{p_k} = (s^\gamma_{p_k}, s^\beta_{p_k})$ according to Eq. (5) and N_{p_k}, σ_{p_k};
7: Compute $\tilde{B}_x = g_{\theta_k}(B_x, s_{p_k})$;
8: Compute $H_k \leftarrow G_{\theta_k}(\tilde{B}_x)$;
9: Send H_k to the active party;
10: ▷ *The active party performs:*
11: Obtain a batch B_y from the label $\mathcal{Y}$ related to B_x;
12: $H = \sum_{k=1}^{K} H_k$;
13: Sample the passport tuple $s_a = (s^\gamma_a, s^\beta_a)$ via Eq. (5) and N_a, σ_a ;
14: Compute $\tilde{H} = g_\omega(H, s_a)$;
15: Compute cross-entropy loss: $\tilde{\ell} = \ell(F_\omega(\tilde{H}), B_y)$
16: Update the active model as: $\omega = \omega - \eta\nabla_\omega\tilde{\ell}$;
17: **for** k in $\{1, \ldots, K\}$ **do**:
18: Compute and send $\nabla_{H_k}\tilde{\ell}$ to each passive party k;
19: ▷ *Passive parties perform:*
20: **for** Passive Party $k \in \{1, \ldots, K\}$ **do**:
21: Update θ_k by $\theta_k = \theta_k - \eta[\nabla_{H_k}\tilde{\ell}][\nabla_{\theta_k}H_k]$
return $\theta_1, \cdots, \theta_K, \omega$

where $G'_{\theta_k}()$ is the composite function $G_{\theta_k} \cdot g_{\theta_k}()$, $k = 1, \cdots, K$.

Proposition 1 illustrates that FedAdOb is trained with the tuple {(Feature, passport), label} = $\{(x, s), y\}$ (see proof in Appendix B). Therefore, the training of FedAdOb can be divided two optimization steps. One is to maximize model performance with the data (x, y) when the passport s is fixed (including W path). The second is also to maximize model performance with the passport s when the data is fixed, i.e., the adaptive obfuscation is also optimized towards maximizing model performance (including γ, β path). Specifically, we take the Eq. (6) as one example, the the derivative for g w.r.t. $W = \theta$ has the following three backpropagation paths via β, γ, θ:

$$\frac{\partial g}{\partial \theta} = \begin{cases} x_{in} \otimes diag(\gamma)^T + \beta & \theta \text{ path} \\ (\theta x_{in})^T \dfrac{\partial \gamma}{\partial \theta} & \gamma \text{ path} \\ \dfrac{\partial \beta}{\partial \theta}, & \beta \text{ path} \end{cases} \tag{9}$$

反向传播路径

where $\otimes$ represents Kronecker product.

5 Security Analysis

We investigate the privacy-preserving capability of FedAdOb against feature reconstruction attacks and label inference attacks. We conduct the privacy analysis with linear regression models, for the sake of brevity. Proofs are deferred to Appendix C.

Definition 1. *Define the forward function of the bottom model G and the top model F:*

- *For the bottom layer: $H = G(x) = W_p s_p^\gamma \cdot W_p x + W_p s_p^\beta$;*
- *For the top layer: $y = F(H) = W_a s_a^\gamma \cdot W_a H + W_a s_a^\beta$.*

where W_p, W_a are 2D matrices of the bottom and top models; $\cdot$ denotes the inner product, s_p^γ, s_p^β are passports embedding into the bottom layers, s_a^γ, s_a^β are passports embedding into the top layers.

5.1 Hardness of Feature Restoration with FedAdOb

Consider the two strong feature restoration attacks, White-box Gradient Inversion (WGI) [1] and White-box Model Inversion (WMI) attack [6], [24], which aims to recover features $\hat{x}$ approximating original features x according to the model gradients and outputs respectively. Specifically, for WMI, the attacker knows the bottom model parameters W_p, forward embedding H, and the way of embedding passports, but does not know the passport. For WGI, the adversary knows the bottom model gradients ∇W_p, and the way of embedding passports, but does not know the passport.

Theorem 1. *Suppose the client protects features x by inserting the s_β^p. The probability of recovering features by the attacker via WGI and WMI attack is at most $\frac{\pi^{m/2}\epsilon^m}{\Gamma(1+m/2)N^m}$ such that the recovering error is less than ϵ, i.e., $\|x - \hat{x}\|_2 \leq \epsilon$, where m denotes the dimension of the passport via flattening, N denotes the passport mean range formulated in Eq. (5) and $\Gamma(\cdot)$ denotes the Gamma distribution.*

Theorem 1 demonstrates that the attacker's probability of recovering features within error ϵ is exponentially small in the dimension of passport size m. The successful recovering probability is inversely proportional to the passport mean range N to the power of m.

5.2 Hardness of Label Recovery with FedAdOb

Consider the passive model competition attack [7] that aims to recover labels owned by the active party. The attacker (i.e., the passive party) leverages a small auxiliary labeled dataset $\{x_i, y_i\}_{i=1}^{n_a}$ belonging to the original training data to train the attack model W_{att}, and then infer labels for the test data. Note that the attacker knows the trained passive model G and forward embedding $H_i = G(x_i)$. Therefore, the attacker optimizes the attack model W_{att} by minimizing $\sum_{i=1}^{n_a} \|W_{att} H_i - \vec{y}_i\|_2{}^2$.

Assumption 1. *Suppose the original main algorithm of VFL achieves zero training loss. For the attack model, we assume the error of the optimized attack model W_{att}^* on test data $\tilde{\ell}_t$ is larger than that of the auxiliary labeled dataset $\tilde{\ell}_a$.*

Theorem 2. *Suppose the active party protects y by embedding s_γ^a, and adversaries aim to recover labels on the test data with the error $\tilde{\ell}_t$ satisfying:*

$$\tilde{\ell}_t \geq \min_{W_{att}} \sum_{i=1}^{n_a} \|(W_{att} - T_i) H_i\|_2, \tag{10}$$

where $T_i = diag(W_a s_{\gamma,i}^a) W_a$ and $s_{\gamma,i}^a$ is the passport for the label y_i embedded in the active model. Moreover, if $H_{i_1} = H_{i_2} = H$ for any $1 \leq i_1, i_2 \leq n_a$, then

$$\tilde{\ell}_t \geq \frac{1}{(n_a - 1)} \sum_{1 \leq i_1 < i_2 \leq n_a} \|(T_{i_1} - T_{i_2}) H\|_2. \tag{11}$$

Proposition 2. *Since passports are randomly generated and W_a and H are fixed, if the $W_a = I, H = \vec{1}$, then it follows that:*

$$\tilde{\ell}_t \geq \frac{1}{(n_a - 1)} \sum_{1 \leq i_1 < i_2 \leq n_a} \|s_{\gamma,i_1}^a - s_{\gamma,i_2}^a\|_2). \tag{12}$$

Theorem 2 and Proposition 2 show that the label recovery error $\tilde{\ell}_t$ has a lower bound, which deserves further explanation. First, when passports are randomly generated for all data, i.e., $s_{\gamma,i_1}^a \neq s_{\gamma,i_2}^a$, then a non-zero label recovery error is guaranteed no matter how adversaries attempt to minimize it. The recovery error thus acts as a protective random noise imposed on true labels. Second, the magnitude of the recovery error monotonically increases with the variance σ^2 of the Gaussian distribution passports sample from (in Eq. (5)), which is a crucial parameter to control privacy-preserving capability (see Experiment results in Sect. 6.3) are in accordance with Theorem 2. Third, it is worth noting that the lower bound is based on the training error of the auxiliary data used by adversaries to launch PMC attacks. Given possible discrepancies between the auxiliary data and private labels, e.g., in terms of distributions and the number of dataset samples, the actual recovery error of private labels can be much larger than the lower bound. Again, this strong protection is observed in experiments (see Sect. 6.2).

6 Experiment

We present empirical studies of FedAdOb on various datasets using different model architectures.

2. $\vec{y}_i$ represents the one-hot vector of label y_i; we use the mean square error loss for the convenience of analysis.

6.1 Experiment Setting

6.1.1 Models & Datasets

We conduct experiments on four image datasets and one tabular dataset: *MNIST* [33], *CIFAR10*, *CIFAR100* [34], ModelNet [35] and Criteo [36]. In HFL, we adopt LeNet [37] on MNIST, *AlexNet* [38] on CIFAR10, *ResNet18* [39] on CIFAR100. In VFL, we adopt LeNet on MNIST and ModelNet, adopt AlexNet and ResNet18 on CIFAR10, and Deep & Cross Network (DCN) [36] on Criteo.

6.1.2 Federated Learning Settings

We partition a neural network into a bottom model and a top model for both HFL and VFL settings. In HFL, each client incorporates their passport information into the bottom model to safeguard the features. In VFL, the passive party embeds the passport information into the last layers of the bottom model to protect the features, while the active party inserts the passport information into the last layers of the top model to protect the labels. In HFL, each client possesses both private features and labels. In VFL, the passive party exclusively contributes private features, while the active party solely provides labels. The details of our HFL and VFL scenarios are summarized in TABLE 2. Please refer to Appendix A for details on the experimental settings.

TABLE 2: Models for evaluation in HFL and VFL. # P denotes the number of parties. FC: fully-connected layer. Conv: convolution layer.

Scenario	Model & Dataset	Bottom Model	Top Model	# P
HFL	LeNet-MNIST	1 Conv	2 Conv+ 3 FC	2
	AlexNet-CIAFR10	1 Conv	4 Conv+ 1 FC	2
	ResNet-CIFAR100	1 Conv	16 Conv+ 1 FC	2
VFL	LeNet-MNIST	2 Conv	3 FC	2
	AlexNet-CIFAR10	5 Conv	1 FC	2
	ResNet18-CIAFR10	17 Conv	1 FC	2
	LeNet-ModelNet	2 Conv	3 FC	7
	DCN-Criteo	3 FC	1 FC	2

6.1.3 Privacy Attack Methods

We consider privacy attacks in both HFL and VFL settings. For **HFL**, we investigate the effectiveness of FedAdOb against White-box and Black-box gradient inversion (WGI and BGI) attacks and White-box [1] and Black-box model inversion (WMI and BMI) attacks [24]. For **VFL**, we evaluate the effectiveness of FedAdOb against feature reconstruction attacks (including CAFE attack [24], [6] and Model Inversion (MI) attack [24]) and label inference attacks (including Passive Model Completion (PMC) attack [7], Norm-based scoring attack and Direction-based scoring attack [28]). The details of the attacks are shown in Appendix A.

6.1.4 Baseline Defense Methods

In both HFL and VFL scenarios, we compare FedAdOb with FedAVG [4], the baseline with no defense, and other three defense methods including **Differential Privacy (DP)** [8], **Sparse** [9], and **InstaHide** [11]. Besides, **SplitFed** [10] is used for comparison with FedAdOb in HFL and **Confusional AutoEncoder** (CAE) [30], **Marvell** [28], **GradPerturb** [15] and **LabelDP** [14] are additional baseline defense methods in VFL (see details in Appendix).

6.1.5 Evaluation Metrics

We use data (feature or label) recovery error and main task accuracy [3] to evaluate defense mechanisms. We adopt the ratio of incorrectly labeled samples by a label inference attack to all labeled samples to measure the performance of that label inference attack. We adopt Mean Square Error (MSE) [1] between original images and images recovered by a feature reconstruction attack to measure the performance of that feature reconstruction attack. MSE is widely used to assess the quality of recovered images. A higher MSE value indicates a higher image recovery error. In addition, we leverage Calibrated Averaged Performance (CAP) [40] to quantify the trade-off between main task accuracy and data recovery error. CAP is defined as follows:

Definition 2 (Calibrated Averaged Performance (CAP))**.** *For a given Privacy-Preserving Mechanism $g_s \in \mathcal{G}$ (s denotes the controlled parameter of g, e.g., the sparsification level, noise level, and passport range) and attack mechanism $a \in \mathcal{A}$, the Calibrated Averaged Performance is defined as:*

$$CAP(g_s, a) = \frac{1}{m} \sum_{s=s_1}^{s_m} Acc(g_s, x) * Rerr(x, \hat{x}_s), \qquad (13)$$

where $Acc(\cdot)$ denotes the main task accuracy and $Rerr(\cdot)$ denotes the recovery error between original data x and estimated data $\hat{x}_s$ via attack a.

6.2 Comparison with Other Defending Methods

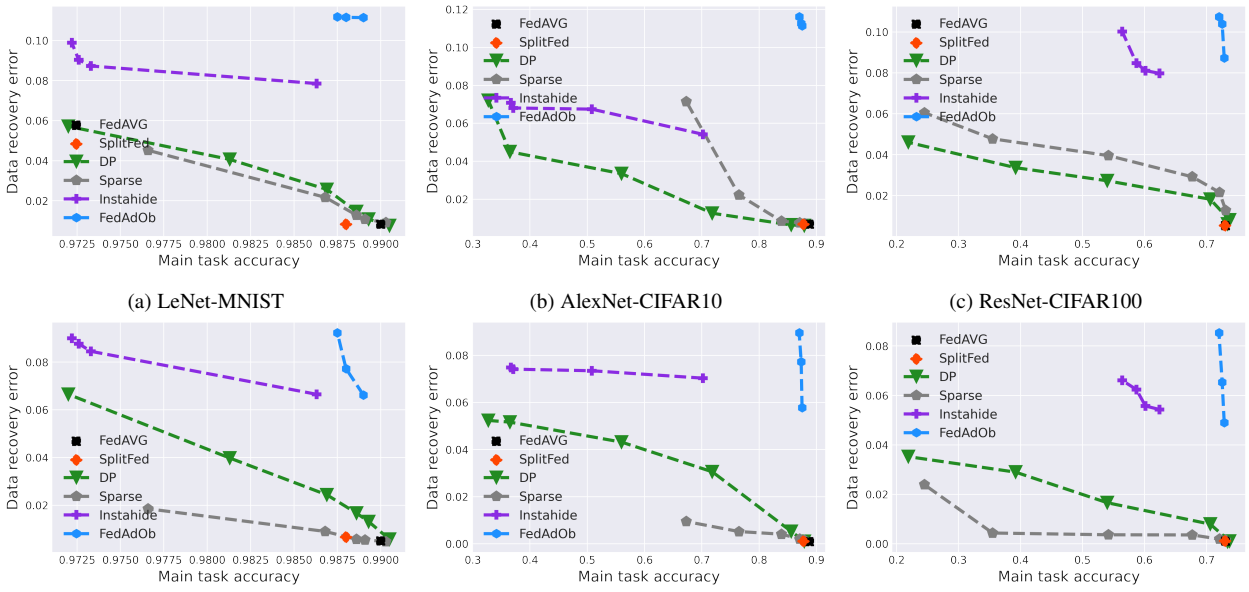
6.2.1 Defending against Feature Restoration Attack in HFL

We evaluate FedAdOb's performance by comparing the averaged clients' accuracy with 5 baselines illustrated in Sect. 6.1. Fig. 3 compares FedAdOb with 4 defense methods in terms of their trade-offs between main task accuracy and data recovery error against WGI and WMI attacks. We have the following three observations:

- FedAdOb facilitates excellent trade-offs between main task accuracy and privacy protection. For instance, model accuracy degradations of FedAdOb are less than 2% for all 6 scenarios shown in Fig. 3 while attackers fail to recover original data (with recovery errors more than 0.045 for WGI attacks and 0.1 for WMI attacks). In contrast, although it achieves high model accuracy, FedAVG suffers from significant privacy leakage.
- The trade-off curves of FedAdOb (dotted dashed line in blue) are near the optimal trade-off towards the top-right corner, and it outperforms all baseline defense methods by large margins.
- Among all baseline defense methods, InstaHide tends to sacrifice a great deal of model accuracy for high privacy-preserving capability. More specifically, for InstaHide, the best cases of performance degradation on CIFAR10 and CIFAR100 are 18.6% and 10.66%, respectively, while the worst cases reach 52.12% and 16.65%, respectively, when mixing up with 4 samples. On the other hand, DP and Sparsification can achieve high model performance but at the cost of high privacy leakage.

In addition, we compare FedAdOb with SplitFed against BGI and BMI attacks. TABLE 3 reports the results, showing that FedAdOb outperforms SplitFed by a large data recovery error on MNIST, CIFAR10, and CIFAR100.

3. AUC is used for binary classification dataset Criteo.



(a) LeNet-MNIST (b) AlexNet-CIFAR10 (c) ResNet-CIFAR100

(d) LeNet-MNIST (e) AlexNet-CIFAR10 (f) ResNet-CIFAR100

Fig. 3: **HFL tradeoff.** Comparison of different defense methods in terms of their trade-offs between main task accuracy and data recovery error against **WGI** attack [1] (the first line) and **WMI** attack [24] (the second line) on LeNet-MNIST, AlexNet-CIFAR10, and ResNet-CIFAR100, respectively. *Trade-off curves near the top right corner are preferred to faraway ones.*

TABLE 3: Data recovery error of SplitFed and FedAdOb ($N = 2$) under **BGI** and **BMI** attacks in **HFL** setting.

Method	Attack	MNIST	CIFAR10	CIFAR100
SplitFed	BGI	0.111±0.002	0.110±0.003	0.087±0.002
	BMI	0.052±0.005	0.028±0.004	0.027±0.004
FedAdOb	BGI	0.111±0.002	0.116±0.002	0.107±0.001
	BMI	0.071±0.003	0.057±0.004	0.055±0.003

6.2.2 *Defending against the Feature Reconstruction Attack in VFL*

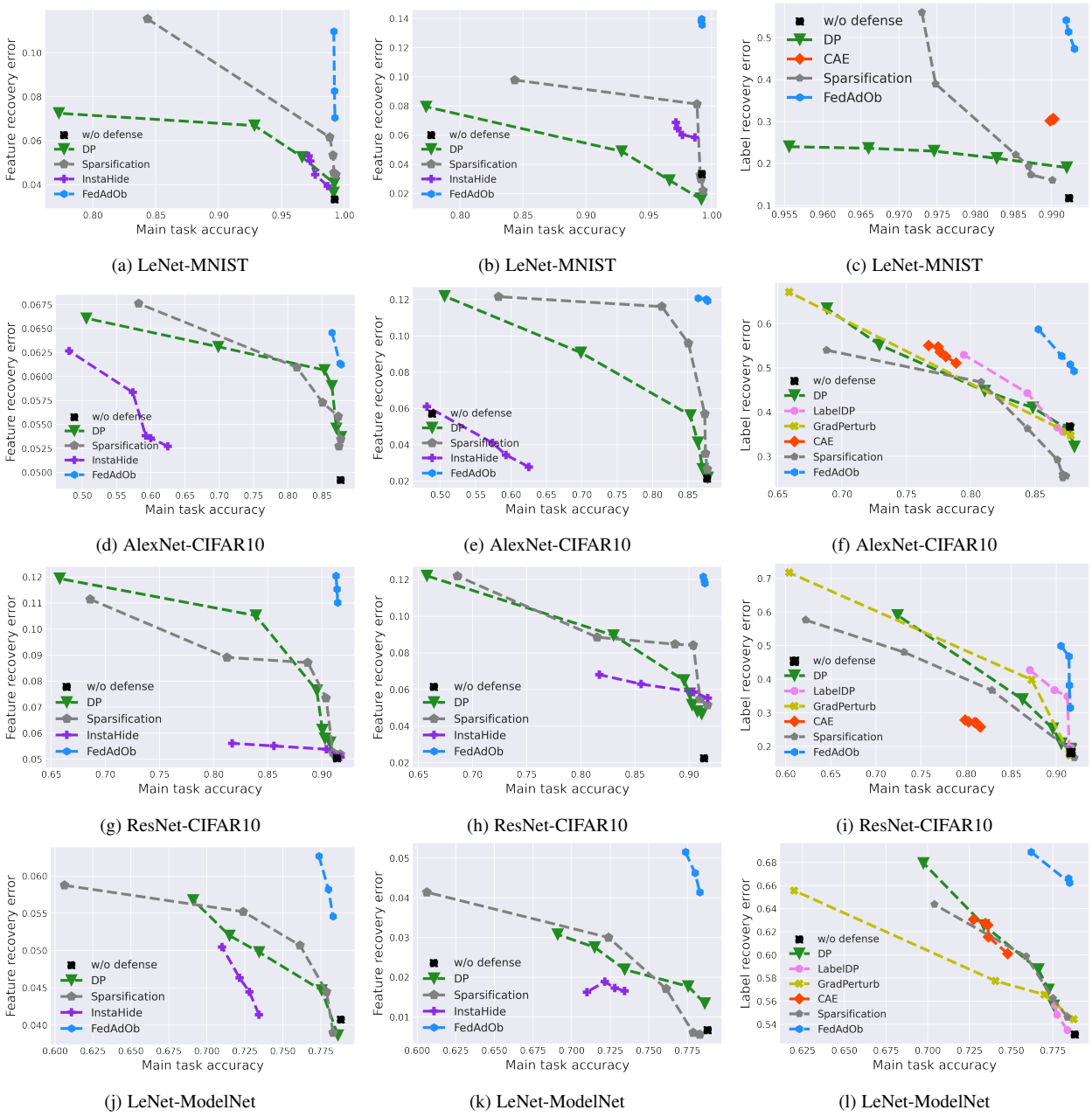
The trade-off between feature recovery error (y-axis) and main task accuracy (x-axis) of FedAdOb, along with baseline methods, is compared in the first and second columns of Fig. 4 against BMI and WMI attacks on four different models. We further compare the recovered images against WMI on Fig. 5. The following observations are made:

- Differential Privacy (DP) and Sparsification methods exhibit a trade-off between high main task performance and low feature recovery error (indicating low privacy leakage). For instance, ResNet-CIFAR10 achieves a main task performance of ≥ 0.90 and a feature recovery error as low as ≤ 0.06 with DP and Sparsification. On the other hand, these methods can achieve a feature recovery error of ≥ 0.11 but obtain a main task performance of ≤ 0.70 for ResNet-CIFAR10.
- InstaHide is generally ineffective against BMI and WMI attacks. Even when more data is mixed in, InstaHide still results in a relatively small feature recovery error but significantly degrades the main task performance.
- The trade-off curves of FedAdOb are located near the top-right corner under both attacks for all models. This indicates that FedAdOb performs best in preserving feature privacy while maintaining the model performance. For example, under MI and CAFE attacks on ResNet-CIFAR10, FedAdOb achieves a main task accuracy of ≥ 0.91 and a feature recovery error of ≥ 0.12. The results presented in Table 1 also demonstrate that FedAdOb offers the best trade-off between privacy and performance under BMI and WMI attacks.
- The reconstructed images under the WMI attack appear as random noise, indicating the successful mitigation of the WMI attack by FedAdOb. Furthermore, the model performance of FedAdOb is nearly indistinguishable from the original model (without defense) across all datasets, as shown in Fig. 4. This demonstrates the superior trade-off achieved by FedAdOb, in stark contrast to existing methods. Notably, approaches such as Differential Privacy (DP) and Sparsification, even at high protection levels ($r5$ and $r7$), result in a significant deterioration of model performance compared to the original model.

6.2.3 *Defending against the Label Inference Attack in VFL*

The third column of Fig. 4 (g)-(i) compares the trade-offs between label recovery error (y-axis) and main task accuracy (x-axis) of FedAdOb with those of baselines in the face of the PMC attack on four models. It is observed DP and its variants (GradPerturb, Label DP), Sparsification, and CAE fail to achieve the goal of obtaining a low level of privacy leakage while maintaining model performance, whereas FedAdOb is more toward the top-right corner, indicating that FedAdOb has a better trade-off between privacy and performance. TABLE 4 reinforces the observation that FedAdOb achieves the best trade-off between privacy and performance under PMC attack.

Furthermore, in the case of NS and DS attacks, which are specifically applied in binary classification problems, we compare the performance of FedAdOb with four other baselines, as depicted in Fig. 6. The results demonstrate that FedAdOb achieves comparable performance to Marvel and outperforms the other three methods.



(a) LeNet-MNIST (b) LeNet-MNIST (c) LeNet-MNIST

(d) AlexNet-CIFAR10 (e) AlexNet-CIFAR10 (f) AlexNet-CIFAR10

(g) ResNet-CIFAR10 (h) ResNet-CIFAR10 (i) ResNet-CIFAR10

(j) LeNet-ModelNet (k) LeNet-ModelNet (l) LeNet-ModelNet

Fig. 4: **VFL tradeoff.** Comparison of different defense methods in terms of their trade-offs between main task accuracy and data (feature or label) recovery error against three attacks on LeNet-MNIST, AlexNet-CIFAR10, ResNet-CIFAR10, respectively and LeNet-ModelNet. **BMI** (the first column) and **WMI** (the second column) are feature reconstruction attacks, whereas **Passive Model Completion** (the third column) is a label inference attack. *A better trade-off curve should be more toward the top-right corner of each figure.*

TABLE 4: The Calibrated averaged performance (CAP) for different defense mechanisms against BMI, WMI, and PMC attacks in **VFL**.

Attack \ Defense		w/o defense	CAE	GradPerturb	LabelDP	Sparsification	DP	InstaHide	FedAdOb
CAFE	LeNet-MNIST	0.033	—	—	—	0.049±0.026	0.033±0.018	0.061±0.004	**0.137±0.002**
	AlexNet-CIFAR10	0.019	—	—	—	0.058±0.026	0.042±0.017	0.023±0.004	**0.105±0.001**
	ResNet-CIFAR10	0.021	—	—	—	0.067±0.014	0.057±0.014	0.053±0.002	**0.109±0.001**
	LeNet-ModelNet	0.005	—	—	—	0.014±0.009	0.016±0.004	0.012±0.001	**0.036±0.003**
MI	LeNet-MNIST	0.033	—	—	—	0.060±0.020	0.049±0.010	0.046±0.005	**0.087±0.001**
	AlexNet-CIFAR10	0.043	—	—	—	0.047±0.003	0.046±0.006	0.032±0.001	**0.054±0.001**
	ResNet-CIFAR10	0.046	—	—	—	0.065±0.012	0.063±0.015	0.047±0.001	**0.105±0.004**
	LeNet-ModelNet	0.032	—	—	—	0.035±0.003	0.036±0.003	0.033±0.002	**0.046±0.002**
PMC	LeNet-MNIST	0.117	0.302±0.002	0.195±0.008	0.314±0.103	0.277±0.140	0.216±0.015	—	**0.506±0.028**
	AlexNet-CIFAR10	0.322	0.415±0.008	0.353±0.063	0.355±0.045	0.283±0.065	0.358±0.051	—	**0.460±0.025**
	ResNet-CIFAR10	0.166	0.217±0.004	0.276±0.119	0.276±0.081	0.268±0.088	0.237±0.087	—	**0.379±0.065**
	LeNet-ModelNet	0.419	0.457±0.004	0.425±0.011	0.426±0.005	0.443±0.011	0.451±0.015	—	**0.523±0.002**

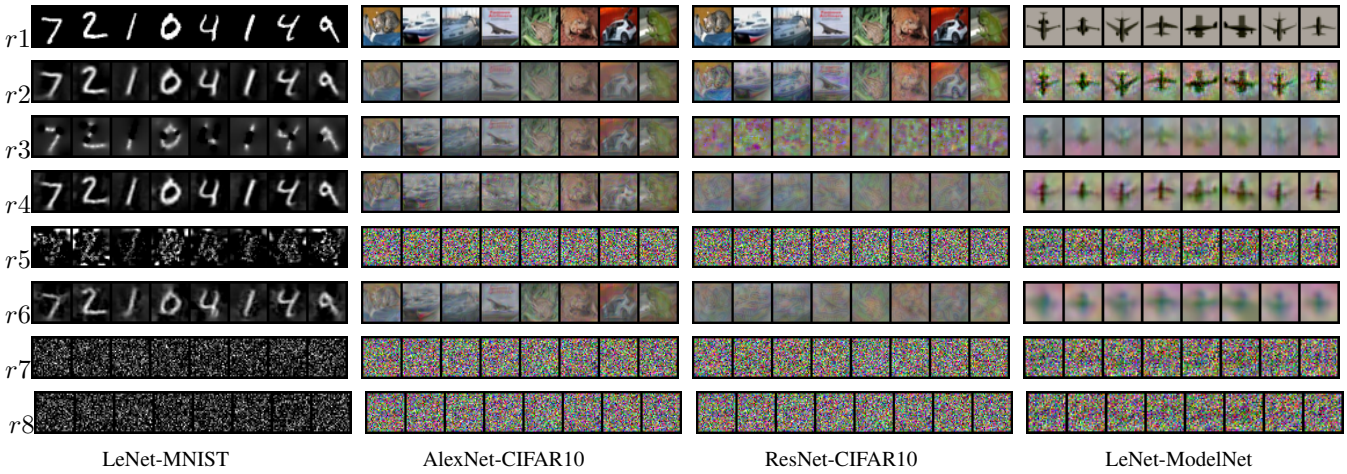
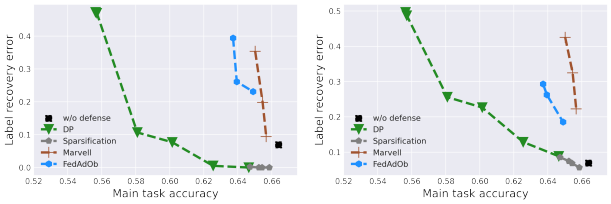


Fig. 5: Original images and images reconstructed by CAFE attack in **VFL** for different defense mechanisms on LeNet-MNIST, AlexNet-CIFAR10, ResNet-CIFAR10 and LeNet-ModelNet respectively. From top to bottom, a row represents original image ($r1$), no defense ($r2$), InstaHide ($r3$), DP with noise level 0.2 ($r4$) and 2 ($r5$), Sparsification with sparsification level 0.5 ($r6$) and 0.05 ($r7$), and FedAdOb ($r8$).



(a) DS (b) NS

Fig. 6: Comparison of different defense methods in terms of their trade-offs between main task accuracy and label recovery error against DS and NS [28] attacks on DCN-Criteo in **VFL**.

6.3 Ablation Study

This section reports ablation study on two critical passport parameters.

As illustrated in main text, when the passports are embedded in a convolution layer or linear layer with c channels[4], for each channel $j \in [c]$, the passport $s(j)$ (the j_{th} element of vector s) is randomly generated as follows:

$$s(j) \sim \mathcal{N}(\mu_j, \sigma^2), \quad \mu_j \in \mathcal{U}(-N, 0), \tag{14}$$

where all $\mu_j, j = 1, \cdots, c$ are different from each other, σ^2 is the variance of Gaussian distribution and N is *the range of passport mean*. σ and N are two crucial parameters determining the privacy-utility trade-off of FedAdOb. This section analyzes how these two parameters influence the data recovery error and main task accuracy for FedAdOb.

6.3.1 Influence of the range of Passport Mean N

For the range of passport mean N, we consider the 2-party VFL scenario in AlexNet-CIFAR10 under feature recovery attack, i.e., CAFE [6], and MI [24] attacks. Fixed the $\sigma = 1$, Table 5 provides the trade-off between main task accuracy and feature recovery error with different passport mean, which illustrates that when the range of passport mean increases, the privacy is preserved at the expense of the minor degradation of model performance less than 1%. This phenomenon is consistent with our analysis of Theorem 1. It is suggested that the N is chosen as 100 when the model performance is less than 0.5%.

4. For the convolution layer, the passport $s \in \mathbb{R}^{c \times h_1 \times h_2}$, where c is channel number, h_1 and h_2 are height and width; for the linear layer, $s \in \mathbb{R}^{c \times h_1}$, where c is channel number, h_1 is height.

N	Main task accuracy	Feature recovery error
1	0.8798	0.0589
2	0.8771	0.0818
5	0.8758	0.1203
10	0.8764	0.1204
50	0.8761	0.1205
100	0.8759	0.1205
200	0.8739	0.1205

TABLE 5: Influence of range of passports mean N on main task accuracy and feature recovery error for FedAdOb against the CAFE attack in **VFL**.

σ^2	Main task accuracy	Label recovery error
0	0.877	0.368
2	0.873	0.510
5	0.870	0.543
10	0.864	0.568
50	0.858	0.577
70	0.844	0.617
100	0.791	0.751

TABLE 6: Influence of passports variance σ^2 on main task accuracy and label recovery error for FedAdOb in **VFL**.

6.3.2 Influence of Passport Variance σ^2

For the passport variance σ^2, we consider the 2-party VFL scenario in AlexNet-CIFAR10 under label recovery attack [7], i.e., PMC attack [7]. Fixed the range of passport mean $N = 100$, Table 6 shows that the main task accuracy decreases and label recovery error increases when the passport variance increases, which is also

verified in Theorem 2. It is suggested that the σ^2 is chosen as 5 when the model performance is less than 0.5%.

6.4 Training and Inference Time

TABLE 7 and 8 tests the training time (for one epoch) and inference time for FedAdOb and baseline defense methods in VFL and HFL setting. It shows that the FedAdOb is as efficient as the w/o defense for both training and inference procedures (the training time on MNIST for each epoch is 7.03s and the inference time is 1.48s in VFL) because embedding passport only introduces a few model parameters to train. It is worth noting that the training time of InstaHide is almost twice that of other methods because InstaHide involves mixing-up multiple feature vectors or labels, which is time-consuming.

TABLE 7: Comparison of training time (for one epoch) and inference time among different defense mechanisms in **VFL**.

Defense Method	LeNet-MNIST		AlexNet-Cifar10		ResNet-Cifar10	
	Train	Infer	Train	Infer	Train	Infer
w/o defense	7.03	1.48	22.37	2.20	22.64	2.18
CAE	7.30	1.48	22.71	2.27	23.02	2.21
Sparsification	6.93	1.45	22.39	2.12	22.61	2.21
DP	7.01	1.49	22.24	2.23	22.63	2.16
InstaHide	21.76	1.50	37.07	2.19	46.26	2.18
FedAdOb (ours)	7.05	1.46	22.58	2.13	22.61	2.16

TABLE 8: Comparison of training time (for one epoch) and inference time among different defense mechanisms in **HFL**.

Defense Method	LeNet-MNIST		AlexNet-Cifar10		ResNet-Cifar100	
	Train	Infer	Train	Infer	Train	Infer
w/o defense	21.30	2.30	69.86	4.95	114.29	9.14
SplitFed	22.05	2.66	70.51	5.18	115.37	9.13
Sparsification	21.76	2.38	70.59	5.34	121.13	9.45
DP	21.53	2.32	70.54	5.63	120.82	9.54
InstaHide	35.62	2.33	100.36	5.17	145.10	9.21
FedAdOb (ours)	23.06	2.23	70.58	5.13	120.41	9.54

7 Conclusion

In this paper, we propose a novel privacy-preserving federated deep learning framework called FedAdOb, which leverages adaptive obfuscation to protect the label and feature simultaneously. Specifically, the proposed adaptive obfuscation is implemented by embedding private passports in the bottom and top models to adapt the deep learning model such that the model performance is preserved. The extensive experiments on multiple datasets and theoretical analysis demonstrate that the FedAdOb can achieve significant improvements over the other protected methods in terms of model performance and privacy-preserving ability.

Besides the privacy attacks from the semi-honest adversary, defending against malicious attacks is another important research direction in federated learning [41], [42]. In the future, we would like to investigate whether FedAdOb is effective in thwarting malicious attacks.

Appendix A
Experimental Setting

This section provides detailed information on our experimental settings. Table 9 summarizes datasets and DNN models evaluated in all experiments and Table 10 summarizes the hyper-parameters used for training models.

A.1 Dataset & Model Architectures

We consider classification tasks by LeNet [37] on MNIST [33], AlexNet [38] on CIFAR10 [43], ResNet [39] on CIFAR10 dataset and LeNet on ModelNet [35], [44]. Specifically, the MNIST database of 10-class handwritten digits has a training set of 60,000 examples, and a test set of 10,000 examples. The CIFAR-10 dataset consists of 60000 32×32 colour images in 10 classes, with 6000 images per class. ModelNet is a widely-used 3D shape classification and shape retrieval benchmark, which currently contains 127,915 3D CAD models from 662 categories. We created 12 2D multi-view images per 3D mesh model by placing 12 virtual cameras evenly distributed around the centroid and partitioned the images into multiple (2 to 6) parties by their angles, which contains 6366 images to train and 1600 images to test. The Criteo dataset is used for predicting ads click-through rate. It has 26 categorical features and 13 continuous features. Categorical features are transformed into embeddings with fixed dimensions before we feed them into the model. In the 2-party VFL setting, both kinds of features are divided into two parts and party A has the labels. The ratio of positive samples to negative samples is $1:16$. To reduce the computational complexity, we randomly select 100000 samples as the training set and 20000 samples as the test set. Moreover, we add the batch normalization layer of the LeNet and AlexNet for after the last convolution layer for existing defense methods.

Scenario	Model & Dataset	Bottom Model	Top Model	# P
HFL	LeNet-MNIST	1 Conv	1 Conv+ 3 FC	10
	AlexNet-CIAFR10	1 Conv	4 Conv+ 1 FC	10
	ResNet-CIFAR100	1 Conv	16 Conv+ 1 FC	10
VFL	LeNet-MNIST	2 Conv	3 FC	2
	AlexNet-CIFAR10	5 Conv	1 FC	2
	ResNet18-CIAFR10	17 Conv	1 FC	2
	LeNet-ModelNet	2 Conv	3 FC	7
	DCN-Criteo	3 FC	1 FC	2

TABLE 9: Models for evaluation in HFL and VFL. # P denotes the number of parties. FC: fully-connected layer. Conv: convolution layer.

A.2 Federated Learning Settings

In HFL, each client incorporates their passport information into the bottom model to safeguard the features. In VFL, we simulate a VFL scenario by splitting a neural network into a bottom model and a top model and assigning the bottom model to each passive party and the top model to the active party. For the 2-party scenario, the passive party has features when the active party owns the corresponding labels. For the 7-party scenario, i.e., the ModelNet, each passive party has two directions of 3D shapes when the active party owns the corresponding label following [44]. Table 9 summarizes our FL scenarios. Also, the VFL framework follows algorithm 1 of [45].

Hyper-parameter	LeNet-MNIST	AlexNet-CIFAR10	ResNet-CIFAR10	LeNet-ModelNet
Optimization method	SGD	SGD	SGD	SGD
Learning rate	1e-2	1e-2	1e-2	1e-2
Weight decay	4e-5	4e-5	4e-5	4e-5
Batch size	64	64	64	64
Iterations	50	100	100	100
The range of N (passport mean)	[1, 50]	[1, 200]	[1, 100]	[1, 50]
The range of σ^2 (passport variance)	[1, 9]	[1, 64]	[1, 25]	[1, 9]

TABLE 10: Hyper-parameters used for training in FedAdOb.

Hyper-parameter	LeNet-MNIST	AlexNet-CIFAR10	ResNet-CIFAR10	LeNet-ModelNet
Optimization method	SGD	SGD	SGD	SGD
Learning rate	1e-2	1e-2	1e-2	1e-2
Weight decay	4e-5	4e-5	4e-5	4e-5
Batch size	64	64	64	64
Iterations	50	100	100	100
The range of N (passport mean)	[1, 50]	[1, 200]	[1, 100]	[1, 50]
The range of σ^2 (passport variance)	[1, 9]	[1, 64]	[1, 25]	[1, 9]

TABLE 11: Hyper-parameters used for training in FedAdOb.

A.3 Privacy Attack Methods

We investigate the effectiveness of FedAdOb on three attacks designed for VFL

- Passive Model Completion (PMC) attack [7] is the label inference attack. For each dataset, the attacker leverages some auxiliary labeled data (40 for MNIST and CIFAR10, 366 for ModelNet) to train the attack model and then the attacker predicts labels of the test data using the trained attack model.
- CAFE attack [6] is a feature reconstruction attack. We assume the attacker knows the forward embedding H and passive party's model G_θ, and we follow the white-box model inversion step (i.e., step 2) of CAFE to recover private features owned by a passive party.
- Model Inversion (MI) attack is a feature reconstruction attack. We follow the black-box setting of [24], in which the attacker does not know the passive model. For each dataset, attacker leverages some labeled data (10000 for MNIST and CIFAR10, 1000 for ModelNet) to train a shallow model approximating the passive model, and then they use the forward embedding and the shallow model to infer the private features of a passive party inversely.

For CAFE and MI attacks, we add the total variation loss [6] into two feature recovery attacks with regularization parameter $\lambda = 0.1$.

A.4 Baseline Defense Methods

The details of baseline defense methods compared with FedAdOb is as follows:

- For **Differential Privacy (DP)** [8], we experiment with Gaussian noise levels ranging from 5e-5 to 1.0. We add noise to gradients for defending against MC attack while add noise to forward embeddings for thwarting CAFE and MI attacks.
- For **Sparsification** [7], [9], we implement gradient sparsification [7] and forward embedding sparsification [9] for defending against label inference attack and feature reconstruction attack, respectively. Sparsification level are chosen from 0.1% to 50.0%.
- For **InstaHide** [11], we mix up 1 to 4 of images to trade-off privacy and utility.
- For **Confusional AutoEncoder** (CAE) [30], we follow the implementation of the original paper. That is, both the encoder and decoder of CAE have the architecture of 2 FC layers. Values of the hyperparameter that controls the confusion level are chosen from 0.0 to 2.0.

A.4.1 Implementation details of FedAdOb

For **FedAdOb**, Passports are embedded in the last convolution layer of the passive party's model and the first fully connected layer of the active party's model. Table 10 summarizes the hyper-parameters for training FedAdOb.

A.5 Notations

APPENDIX B
FORMULATION OF FEDADOB IN HFL AND VFL

Consider a neural network $f_\Theta(x) : \mathcal{X} \to \mathbb{R}$, where $x \in \mathcal{X}$, Θ denotes model parameters of neural networks.

VFL. K passive parties and one active party collaboratively optimize $\Theta = (\omega, \theta_1, \cdots, \theta_K)$ of network according to Eq. (1).

$$\min_{\omega,\theta_1,\cdots,\theta_K} \frac{1}{n}\sum_{i=1}^{n} \ell(F_\omega \circ (G_{\theta_1}(x_{1,i}), G_{\theta_2}(x_{2,i}), \cdots, G_{\theta_K}(x_{K,i})), y_i), \tag{1}$$

where ℓ is the loss, e.g., the cross-entropy loss, passive party P_k owns features $\mathcal{D}_k = (x_{k,1}, \cdots, x_{k,n}) \in \mathcal{X}_k$ and the passive model G_{θ_k}, the active party owns the labels $y \in \mathcal{Y}$ and active

Notation	Meaning
F_ω, G_{θ_k}	Active model and k_{th} passive model
W_p, W_a, W_{att}	The 2D matrix of the active model, passive model and attack model for the regression task
s_a, s_p	The passport of the active model and passive model
$s_a^\gamma, s_a^\beta, s_p^\gamma, s_p^\beta$	The passport of the active model and passive model w.r.t γ and β respectively
γ, β	The scaling and bias
g_W	Adaptive obfuscations w.r.t model parameters W
H_k	Forward embedding passive party k transfers to the active party
$\tilde{\ell}$	the loss of main task
$\nabla_{H_k}\tilde{l}$	Backward gradients the active party transfers to the passive party k
$\mathcal{D}_k = \{x_{k,i}\}_{i=1}^{n_i}$	n_i private features of passive party k
y	Private labels of active party
K	The number of party
N	The range of Gaussian mean of passports sample
σ^2	The variance of passports sample
$\tilde{\ell}_a$	Training error on the auxiliary dataset for attackers
$\tilde{\ell}_t$	Test error on the test dataset for attackers
n_a	The number of auxiliary dataset to do PMC attack
T	Number of optimization steps for VFL
η	learning rate
$\|\cdot\|$	ℓ_2 norm

TABLE 12: Table of Notations

model F_ω, $\mathcal{X}_k$ and $\mathcal{Y}$ are the feature space of party P_k and the label space respectively. Furthermore, FedAdOb aims to optimize:

$$\min_{\omega,\theta_1,\cdots,\theta_K} \frac{1}{N}\sum_{i=1}^{N} \ell(F_\omega g_\omega \circ (G_{\theta_1}(g_{\theta_1}(x_{1,i}, s_{p_1})), \cdots, G_{\theta_K}(g_{\theta_K}(x_{K,i}, s_{p_K})), y_i). \tag{2}$$

Denote the composite function $G_{\theta_j} g_{\theta_j}()$ as $G'_{\theta_j}()$, $j = 1, \cdots, K$. Then we rewrite the Eq. (2) as follows

$$\min_{\omega,\theta_1,\cdots,\theta_K} \frac{1}{N}\sum_{i=1}^{N} \ell(F'_\omega \circ (G'_{\theta_1}(x_{1,i}, s^{p_1})), \cdots, G'_{\theta_K}(x_{K,i}, s^{p_K})), s^a, y_i), \tag{3}$$

HFL. K party collaboratively optimize $\Theta = (\omega, \theta_1, \cdots, \theta_K)$ of network according to Eq. (4).

$$\min_{\theta_1,\cdots,\theta_K,\omega} \sum_{k=1}^{K}\sum_{i=1}^{n_k} \frac{\ell(F_\omega(x_{k,i}), y_{k,i})}{n_1 + \cdots + n_K}, \tag{4}$$

where ℓ is the loss, e.g., the cross-entropy loss, $\mathcal{D}_k = \{(x_{k,i}, y_{k,i})\}_{i=1}^{n_k}$ is the dataset with size n_k owned by client k. Furthermore, FedAdOb aims to optimize:

$$\min_{\omega,\theta_1,\cdots,\theta_K} \frac{1}{N}\sum_{i=1}^{N} \ell(F_\omega g_\omega \circ (G_{\theta_1}(g_{\theta_1}(x_{1,i}, s_{p_1})), \cdots, G_{\theta_K}(g_{\theta_K}(x_{K,i}, s_{p_K})), y_i). \tag{5}$$

Denote the composite function $G_{\theta_j} g_{\theta_j}()$ as $G'_{\theta_j}()$, $j = 1, \cdots, K$. Then we rewrite the Eq. (5) as follows

$$\min_{\theta_1,\cdots,\theta_K,\omega} \sum_{k=1}^{K}\sum_{i=1}^{n_k} \frac{\ell(F_\omega \circ (G'_{\theta_k}(x_{k,i}, s_{p_k}), y_{k,i})}{n_1 + \cdots + n_K}, \tag{6}$$

Then Proposition 1 ends the proof.

APPENDIX C
SECURITY ANALYSIS FOR FEDADOB

We investigate the privacy-preserving capability of FedAdOb against feature reconstruction attack and label inference attack. Note that we conduct the privacy analysis with linear regression models, for the sake of brevity.

Definition 1. *Define the forward function of the bottom model G and the top model F:*

- *For the bottom layer: $H = G(x) = W_p s_p^\gamma \cdot W_p x + W_p s_p^\beta$.*
- *For the top layer: $y = F(H) = W_a s_a^\gamma \cdot W_a H + W_a s_a^\beta$.*

where W_p, W_a are 2D matrices of the bottom and top models; $\cdot$ denotes the inner product, s_p^γ, s_p^β are passports embedding into the bottom layers, s_a^γ, s_a^β are passports embedding into the top layers.

C.1 Hardness of Feature Restoration with FedAdOb

Considering two strong feature restoration attacks, White-box Gradient Inversion (WGI) [1] and White-box Model Inversion (WMI) attack [6], [24], which aims to recover features $\hat{x}$ approximating original features x according to the model gradients and outputs respectively. Specifically, for WMI, the attacker knows the bottom model parameters W_p, forward embedding H and the way of embedding passport, but does not know the passport. For WGI, the adversary knows the bottom model gradients ∇W_p, and the way of embedding passport, but does not know the passport.

Lemma 1. *Suppose a bottom model as the Def. 1 illustrates, an attack estimates the private feature by guessing passports $s_{\gamma'}^p$ and $s_p^{\beta'}$ via WMI attack. If W_p is invertible, we could obtain the difference between x and estimated $\hat{x}$ by the adversary in the following two cases:*

- *When inserting the s_γ^p for the passive party,*

$$\|x - \hat{x}\|_2 \geq \frac{\|(D_\gamma^{-1} - D_{\gamma'}^{-1})H\|_2}{\|W_p\|_2}. \tag{7}$$

- *When inserting the s_p^β for the passive party,*

$$\|x - \hat{x}\|_2 = \|s_p^\beta - s_p^{\beta'}\|_2, \tag{8}$$

where $D_\gamma = diag(W_p s_\gamma^p), D_{\gamma'} = diag(W_p s_{\gamma'}^p), D_\beta = diag(W_p s_p^\beta), D_{\beta'} = diag(W_p s_p^{\beta'})$ and $W_p^\dagger$ is the Moore–Penrose inverse of W_p.

Proof. For inserting the s_γ^p,

$$H = W_p s_\gamma^p * W_p x.$$

The attacker with knowing W_p has

$$H = W_p s_{\gamma'}^p * W_p \hat{x}.$$

Denote $D_\gamma = diag(W_p s_\gamma^p), D_{\gamma'} = diag(W_p s_{\gamma'}^p)$. If A is invertible, due to

$$\|A\|_2 \|A^{-1}B\|_2 \leq \|AA^{-1}B\|_2 = \|B\|_2,$$

we have

$$\|A^{-1}B\|_2 \geq \frac{\|B\|_2}{\|A\|_2}.$$

Therefore, if $W_P^\dagger$ is the Moore–Penrose inverse of W_p and W_p is invertible, we can obtain

$$\begin{aligned}\|\hat{x}-x\| &= \|W_p^\dagger D_\gamma^{-1}H - W_p^\dagger D_{\gamma'}^{-1}H\|_2 \\ &= \|W_p^\dagger (D_\gamma^{-1} - D_{\gamma'}^{-1})H\|_2 \\ &\geq \frac{\|(D_\gamma^{-1} - D_{\gamma'}^{-1})H\|_2}{\|W_p\|_2}.\end{aligned} \tag{9}$$

On the other hand, for inserting the s_p^β of passive party,

$$H = W_p x + W_p s_p^\beta.$$

The attacker with knowing W_p has

$$H = W_p x + W_p s_p^{\beta'}.$$

We further obtain

$$\begin{aligned}\|\hat{x}-x\| &= \|W_p^\dagger (H - D_\beta) - W_p^\dagger (H - D_{\beta'})\|_2 \\ &= \|W_p^\dagger (W_p s_p^\beta - W_p s_p^{\beta'})\|_2 \\ &= \|s_p^\beta - s_p^{\beta'}\|_2.\end{aligned} \tag{10}$$

□

Remark 1. *If the adversary don't know the existence of passport layer, then $D_{\gamma'} = I$ and $s_p^{\beta'} = 0$ in Eq.* (9) *and* (10)*.*

Remark 2. *Eq.* (8) *only needs W_p is a left inverse, i.e., W_p has linearly independent columns.*

Lemma 2. *Suppose a bottom model as the Def. 1 illustrates, an attack estimates the private feature by guessing passports $s_{\gamma'}^p$ and $s_p^{\beta'}$ via WGI attack. If ∇b is non-zero and inserting the s_p^β for the bottom model, we could obtain*

$$\|x - \hat{x}\|_2 = \|s_p^\beta - s_p^{\beta'}\|_2. \tag{11}$$

Proof. For inserting the s_p^β of passive model,

$$H = W_p x + b_\beta = W_p x + W_p s_p^\beta.$$

The attacker with knowing W_p has

$$H = W_p x + b_{\beta'} = W_p x + W_p s_p^{\beta'}.$$

Denote $b_{\beta'} = W_p s_p^{\beta'}$ and $b_\beta = W_p s_p^\beta$, $\nabla W_p = \nabla_{W_p}\ell$ and $\nabla W_p = \nabla_{W_p}\ell$. According to chain rule, we can obtain

$$\nabla W_p = \nabla_H \ell (x + s_\beta)^T, \tag{12}$$

and

$$\nabla b = \nabla_H \ell I \tag{13}$$

Therefore,

$$\nabla W_p = \nabla b (x + s_p^\beta)^T, \tag{14}$$

and

$$\nabla W_p = \nabla b (\hat{x} + s_p^{\beta'})^T, \tag{15}$$

If ∇b is not zero, then

$$\hat{x} + s_p^{\beta'} = x + s_p^\beta, \tag{16}$$

which implies to

$$\|x - \hat{x}\|_2 = \|s_p^\beta - s_p^{\beta'}\|_2. \tag{17}$$

□

Lemma 1 and Lemma 2 illustrate that the difference between estimated feature and original feature has the lower bound, which depends on the difference between original passport and inferred passport. Specifically, if the inferred passport is far away from $\|s_p^\beta - s_p^{\beta'}\|_2$ and $\|(D_\gamma^{-1} - D_{\gamma'}^{-1})\|$ goes large causing a large reconstruction error by attackers. Furthermore, we provide the analysis of the probability of attackers to reconstruct the x in Theorem 1. Let m denote the dimension of the passport via flattening, N denote the passport mean range formulated in Eq. (5) of the main text and $\Gamma(\cdot)$ denote the Gamma distribution.

Theorem 1. *Suppose the passive party protects features x by inserting the s_p^β. The probability of recovering features by the attacker via white-box MI and GI attack is at most $\frac{\pi^{m/2}\epsilon^m}{\Gamma(1+m/2)N^m}$ such that the recovering error is less than ϵ, i.e., $\|x - \hat{x}\|_2 \leq \epsilon$,*

Proof. According to Lemma 1, the attacker aims to recover the feature $\hat{x}$ within the ϵ error from the original feature x, that is, the guessed passport needs to satisfy:

$$\|s_p^{\beta'} - s_p^\beta\|_2 \leq \epsilon \tag{18}$$

Therefore, the area of inferred passport of attackers is sphere with the center s_p^β and radius ϵ. And the volumes of this area is at most $\frac{\pi^{m/2}\epsilon^m}{\Gamma(1+m/2)}$, where F represent the Gamma distribution and m is dimension of the passport. Consequently, the probability of attackers to successfully recover the feature within ϵ error is:

$$p \leq \frac{\pi^{m/2}\epsilon^m}{\Gamma(1+m/2)N^m},$$

攻击恢复成功的概率上界

where the whole space is $(-N, 0)^m$ with volume N^m. □

Theorem 1 demonstrates that the attacker's probability of recovering features within error ϵ is exponentially small in the dimension of passport size m. The successful recovering probability is inversely proportional to the passport mean range N, which is consistent with our ablation study in Sect. 6.3 in the main text.

Remark 3. *The attacker's behaviour we consider in Theorem 1 is that they guess the private passport randomly.*

C.2 Hardness of Label Recovery with FedAdOb

Consider the passive model competition attack [7] that aims to recover labels owned by the active party. The attacker (i.e., the passive party) leverages a small auxiliary labeled dataset $\{x_i, y_i\}_{i=1}^{n_a}$ belonging to the original training data to train the attack model W_{att}, and then infer labels for the test data. Note that the attacker knows the trained passive model G and forward embedding $H_i = G(x_i)$. Therefore, they optimizes the attack model W_{att} by minimizing $\ell = \sum_{i=1}^{n_a} \|W_{att}H_i - y_i\|_2$.

Assumption 2. *Suppose the original main algorithm of VFL is convergent. For the attack model, we assume the error of the optimized attack model W_{att}^* on test data $\tilde{\ell}_t$ is larger than that of the auxiliary labeled dataset $\tilde{\ell}_a$.*

Remark 4. *The test error is usually higher than the training error because the error is computed on an unknown dataset that the model hasn't seen.*

Theorem 2. *Suppose the active party protect y by embedding s_γ^a, and adversaries aim to recover labels on the test data with the error $\tilde{\ell}_t$ satisfying:*

$$\tilde{\ell}_t \geq \min_{W_{att}} \sum_{i=1}^{n_a} \|(W_{att} - T_i)H_i\|_2, \tag{19}$$

where $T_i = diag(W_a s^a_{\gamma,i}) W_a$ *and* $s^a_{\gamma,i}$ *is the passport for the label* y_i *embedded in the active model. Moreover, if* $H_{i_1} = H_{i_2} = H$ *for any* $1 \le i_1, i_2 \le n_a$*, then*

$$\tilde{\ell}_t \ge \frac{1}{(n_a - 1)} \sum_{1 \le i_1 < i_2 \le n_a} \|(T_{i_1} - T_{i_2})H\|_2 \tag{20}$$

Proof. For only inserting the passport s^a_γ of active party, according to Assumption 2, we have

$$y_i = W_a s^a_\gamma \cdot W_a H_i \tag{21}$$

Moreover, the attackers aims to optimize

$$\begin{aligned}
&\min_W \sum_{i=1}^{n_a} \|W H_i - y_i\|_2 \\
=&\min_W \sum_{i=1}^{n_a} \|W H_i - W_a s^a_\gamma \cdot W_a H_i\|_2 \\
=&\min_W \sum_{i=1}^{n_a} \|(W - T_i) H_i\|_2,
\end{aligned}$$

where $T_i = diag(W_a s^a_{\gamma,i}) W_a$. Therefore, based on Assumption 2, $\tilde{\ell}_t \ge \min_W \sum_{i=1}^{n_a} \|(W - T_i)H_i\|_2$. Moreover, if $H_i = H_j = H$ for any $i_1, i_2 \in [n_a]$, then

$$\begin{aligned}
\tilde{\ell}_t &\ge \min_W \sum_{i=1}^{n_a} \|(W - T_i)H_i\|_2 \\
&= \min_W \frac{1}{2(n_a - 1)} \sum_{1 \le i_1, i_2 \le n_a} (\|(W - T_{i_1})H\|_2 \\
&\quad + \|(W - T_{i_2})H\|_2) \\
&\ge \min_W \frac{1}{2(n_a - 1)} \sum_{1 \le i_1, i_2 \le n_a} (\|(T_{i_1} - T_{i_2})H\|_2) \\
&= \frac{1}{(n_a - 1)} \sum_{1 \le i_1 < i_2 \le n_a} (\|(T_{i_1} - T_{i_2})H\|_2)
\end{aligned}$$

□

Proposition 1. *Since passports are randomly generated and* W_a *and* H *are fixed, if the* $W_a = I, H = \vec{1}$*, then it follows that:*

$$\tilde{\ell}_t \ge \frac{1}{(n_a - 1)} \sum_{1 \le i_1 < i_2 \le n_a} \|s^a_{\gamma,i_1} - s^a_{\gamma,i_2}\|_2) \tag{22}$$

Proof. When $W_a = I$ and $H = \vec{1}$, $(T_{i_1} - T_{i_2})H = s^a_{\gamma,i_1} - s_{\gamma,i_2}$. Therefore, based on Theorem 2, we obtain

$$\tilde{\ell}_t \ge \frac{1}{(n_a - 1)} \sum_{1 \le i_1 < i_2 \le n_a} \|s^a_{\gamma,i_1} - s^a_{\gamma,i_2}\|_2) \tag{23}$$

□

Theorem 2 and Proposition 1 show that the label recovery error $\tilde{\ell}_t$ has a lower bound, which deserves further explanations.

- First, when passports are randomly generated for all data, i.e., $s^a_{\gamma,i_1} \ne s^a_{\gamma,i_2}$, then a non-zero label recovery error is guaranteed no matter how adversaries attempt to minimize it. The recovery error thus acts as a protective random noise imposed on true labels.
- Second, the magnitude of the recovery error monotonically increases with the variance σ^2 of the Gaussian distribution passports sample from (It is because the difference of two samples from the same Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ depends on the variance σ^2), which is a crucial parameter to control privacy-preserving capability. Experiments on Sect. 6.4 of the main text also verify this phenomenon.
- Third, it is worth noting that the lower bound is based on the training error of the auxiliary data used by adversaries to launch PMC attacks. Given possible discrepancies between the auxiliary data and private labels, e.g., in terms of distributions and the number of dataset samples, the actual recovery error of private labels can be much larger than the lower bound.

REFERENCES

[1] L. Zhu, Z. Liu, , and S. Han, "Deep leakage from gradients," in *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2019.

[2] J. Konečnỳ, B. McMahan, and D. Ramage, "Federated optimization: Distributed optimization beyond the datacenter," *arXiv preprint arXiv:1511.03575*, 2015.

[3] J. Konečnỳ, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.

[4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.

[5] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.

[6] X. Jin, P.-Y. Chen, C.-Y. Hsu, C.-M. Yu, and T. Chen, "Cafe: Catastrophic data leakage in vertical federated learning," *NeurIPS*, vol. 34, pp. 994–1006, 2021.

[7] C. Fu, X. Zhang, S. Ji, J. Chen, J. Wu, S. Guo, J. Zhou, A. X. Liu, and T. Wang, "Label inference attacks against vertical federated learning," in *31st USENIX Security Symposium (USENIX Security 22), Boston, MA*, 2022.

[8] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 308–318.

[9] Y. Lin, S. Han, H. Mao, Y. Wang, and B. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *International Conference on Learning Representations*, 2018.

[10] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "Splitfed: When federated learning meets split learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 8, 2022, pp. 8485–8493.

[11] Y. Huang, Z. Song, K. Li, and S. Arora, "Instahide: Instance-hiding schemes for private distributed learning," in *International conference on machine learning*. PMLR, 2020, pp. 4507–4518.

[12] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *International Conference on Learning Representations*, 2018.

[13] Y. Liu, Z. Yi, Y. Kang, Y. He, W. Liu, T. Zou, and Q. Yang, "Defending label inference and backdoor attacks in vertical federated learning," *arXiv preprint arXiv:2112.05409*, 2021.

[14] B. Ghazi, N. Golowich, R. Kumar, P. Manurangsi, and C. Zhang, "Deep learning with label differential privacy," *Advances in neural information processing systems*, vol. 34, pp. 27 131–27 145, 2021.

[15] X. Yang, J. Sun, Y. Yao, J. Xie, and C. Wang, "Differentially private label protection in split learning," *arXiv preprint arXiv:2203.02073*, 2022.

[16] N. Dryden, T. Moon, S. A. Jacobs, and B. Van Essen, "Communication quantization for data-parallel training of deep neural networks," in *2016 2nd Workshop on Machine Learning in HPC Environments (MLHPC)*. IEEE, 2016, pp. 1–8.

[17] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2017, pp. 440–445.

[18] Y. Liu, Y. Kang, C. Xing, T. Chen, and Q. Yang, "A secure federated transfer learning framework," *IEEE Intelligent Systems*, vol. 35, no. 4, pp. 70–82, 2020.

[19] X. Zhang, Y. Kang, K. Chen, L. Fan, and Q. Yang, "Trading off privacy, utility and efficiency in federated learning," *ACM Trans. Intell. Syst. Technol.*, may 2023.

[20] Y. Kang, J. Luo, Y. He, X. Zhang, L. Fan, and Q. Yang, "A framework for evaluating privacy-utility trade-off in vertical federated learning," *arXiv preprint arXiv:2209.03885*, 2022.

[21] L. Fan, K. W. Ng, C. S. Chan, and Q. Yang, "Deepip: Deep neural network intellectual property protection with passports," *IEEE Transactions on Pattern Analysis & Machine Intelligence*, no. 01, pp. 1–1, 2021.

[22] B. Li, L. Fan, H. Gu, J. Li, and Q. Yang, "Fedipr: Ownership verification for federated deep neural network models," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[23] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients-how easy is it to break privacy in federated learning?" *Advances in Neural Information Processing Systems*, vol. 33, pp. 16 937–16 947, 2020.

[24] Z. He, T. Zhang, and R. B. Lee, "Model inversion attacks against collaborative inference," in *Proceedings of the 35th Annual Computer Security Applications Conference*, 2019, pp. 148–162.

[25] H. Gu, L. Fan, B. Li, Y. Kang, Y. Yao, and Q. Yang, "Federated deep learning with bayesian privacy," *arXiv preprint arXiv:2109.13012*, 2021.

[26] S. Hardy, W. Henecka, H. Ivey-Law, R. Nock, G. Patrini, G. Smith, and B. Thorne, "Private federated learning on vertically partitioned data via entity resolution and additively homomorphic encryption," *arXiv preprint arXiv:1711.10677*, 2017.

[27] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, p. 612–613, nov 1979. [Online]. Available: https://doi.org/10.1145/359168.359176

[28] O. Li, J. Sun, X. Yang, W. Gao, H. Zhang, J. Xie, V. Smith, and C. Wang, "Label leakage and protection in two-party split learning," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=cOtBRgsf2fO

[29] K. Cheng, T. Fan, Y. Jin, Y. Liu, T. Chen, D. Papadopoulos, and Q. Yang, "Secureboost: A lossless federated learning framework," *IEEE Intelligent Systems*, vol. 36, no. 06, pp. 87–98, nov 2021.

[30] T. Zou, Y. Liu, Y. Kang, W. Liu, Y. He, Z. Yi, Q. Yang, and Y.-Q. Zhang, "Defending batch-level label inference and replacement attacks in vertical federated learning," *IEEE Transactions on Big Data*, 2022.

[31] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," *Advances in neural information processing systems*, vol. 32, 2019.

[32] L. Fan, K. W. Ng, and C. S. Chan, "Rethinking deep neural network ownership verification: Embedding passports to defeat ambiguity attacks," *Advances in neural information processing systems*, vol. 32, 2019.

[33] Y. LeCun, C. Cortes, and C. Burges, "Mnist handwritten digit database," *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, vol. 2, 2010.

[34] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," *Master's thesis, Department of Computer Science, University of Toronto*, 2009.

[35] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3d shapenets: A deep representation for volumetric shapes," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1912–1920.

[36] R. Wang, B. Fu, G. Fu, and M. Wang, "Deep & cross network for ad click predictions," in *Proceedings of the ADKDD'17*, 2017, pp. 1–7.

[37] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[38] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

[39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[40] L. Fan, K. W. Ng, C. Ju, T. Zhang, C. Liu, C. S. Chan, and Q. Yang, "Rethinking privacy preserving deep learning: How to evaluate and thwart privacy attacks," in *Federated Learning*. Springer, 2020, pp. 32–50.

[41] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," *Advances in neural information processing systems*, vol. 30, 2017.

[42] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2020, pp. 2938–2948.

[43] A. Krizhevsky, V. Nair, and G. Hinton, "The cifar-10 dataset," *online: http://www. cs. toronto. edu/kriz/cifar. html*, vol. 55, no. 5, 2014.

[44] Y. Liu, X. Liang, J. Luo, Y. He, T. Chen, Q. Yao, and Q. Yang, "Cross-silo federated neural architecture search for heterogeneous and cooperative systems," in *Federated and Transfer Learning*. Springer, 2022, pp. 57–86.

[45] Y. Liu, Y. Kang, T. Zou, Y. Pu, Y. He, X. Ye, Y. Ouyang, Y.-Q. Zhang, and Q. Yang, "Vertical federated learning," *arXiv preprint arXiv:2211.12814*, 2022.